Attention-Based Encoder-Decoder Models: Lecture Walkthrough

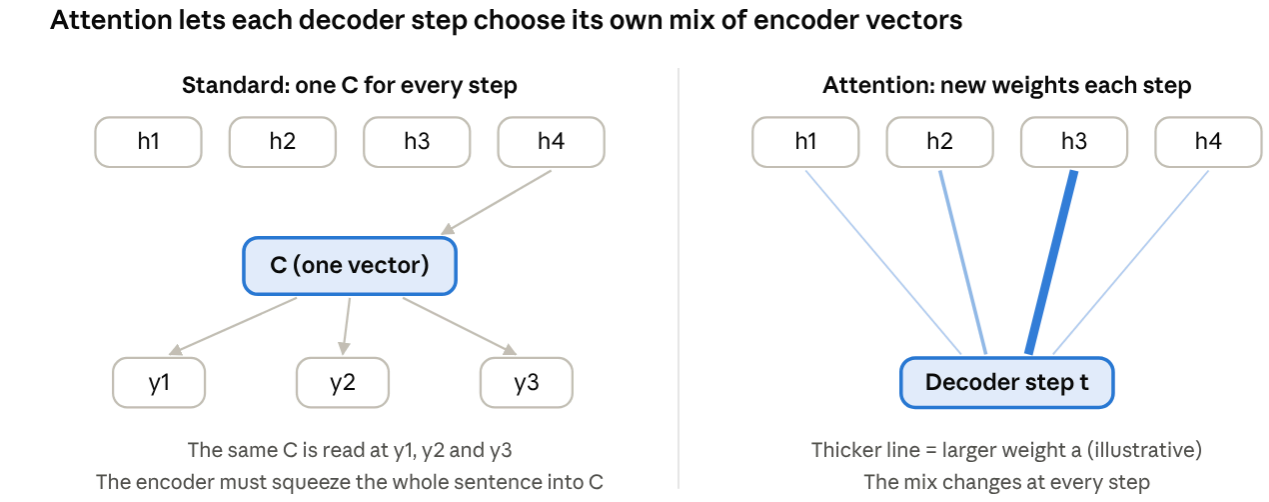
A sentence-by-sentence study guide to the lecture, in plain language.

Attention lets the decoder pick a different mix of encoder vectors at every output step, instead of relying on one fixed vector. This guide follows the lecture in order, point by point, with a drawing for each stage.

Part 1: Limits of the standard encoder-decoder

The standard model struggles with long sentences because one fixed-length vector has to carry everything the decoder needs.

- In a very long sentence, it becomes very difficult for the standard encoder-decoder model to compress all the necessary information for the decoder into a single fixed-length vector.
- The same context vector is used at all stages of the decoding process.
- So the encoder is under pressure to compress all the necessary information of the source sentence into that one vector.
- If the number of words in the input is very high, compressing everything into a fixed-length vector is a hard job. This is the issue identified with the standard encoder-decoder model.



standard vs attention encoder-decoder · schematic

Left: every decoder step (y1, y2, y3) reads the same C. Right: one decoder step weighs each encoder vector differently.

Part 2: Learning to align and translate

The fix comes from asking how people translate, and the answer is attention.

- The lecture's hint: suppose you translate from language A to language B and have produced 3 words. To come up with the 4th word, do you need the context of the entire sentence, or do you selectively attend?
- You attend to the words you need to look at to produce the next word. That is where the word attention comes from.
- The new philosophy is called learning to align and translate.
- Each time the decoder generates a word, it searches for a set of positions in the source sentence where the most relevant information is concentrated. (The slide underlines the most important words.)
- Translation happens one word at a time. When the decoder is about to produce the next word, the model can look at the positions in the source that matter and pick those contexts for this point in time.
- The context needed at each decoding step need not be the same. The model can select different contexts at different points of the decoding process.
- This eases the burden on the encoder, which no longer has to compress everything into one fixed vector.
- The model predicts a target word from the context vectors tied to those source positions and from all previously generated target words.
- The context vector is not the same at every step. It changes, and the model attends to the context vectors that matter at that moment.

In short, the strategy improves how the model handles long sentences and gives a more nuanced translation, because it attends to different parts of the source as needed.

Part 3: Example, translating English to Python

The lecture uses English to Python because it is also a translation task: one side is a natural language and the other is a programming language.

The task

- The input sentence is "Python program to print Hello world", the first piece of code people write in Python.
- The goal is a model that takes this English statement X and generates Python code that does exactly that: `print("hello world")`.
- The model is an encoder-decoder. The encoder does not change: its job is to build contexts from the input sequence and pass them to the decoder.

What the encoder produces

1. Start with an initial vector h1.
2. The first word, "Python", enters as a vector embedding. Applying function F gives h2.
3. F applied to "program" gives h3, and so on until F is applied to the embedding of "world".
4. The result is seven vectors, h1 to h7. The lecturer first said h8 and corrected it: it is h7.
5. The same RNN function F (matrix operations) is used at every step. That is a key property of an RNN.
6. In a standard encoder-decoder, the last vector is supposed to hold all the information about the sentence, and decoding starts from it.

Do we need the whole sentence for every word?

- The correct translation is `print`, then `hello world`. You can type this command in a Python interpreter and it prints the statement.
- Question: to produce the first token `print`, do you need to know what is going to be printed? The answer is no. The model generates one word at a time, and what to print comes after `print` has been written.
- This is the attention mechanism: when emitting `print`, focus only on certain parts of the source. The most important are "Python", "program" and "print". The least important are "hello" and "world".
- To generate `hello`, the decoder should put a lot of weight on the word "hello" compared with "program" or "print". To generate `world`, it should put more weight on "world".
- With the standard model, the decoder has only a single vector C and has to produce `print`, `hello` and `world` from it. That is a difficult task in longer sentences.
- With attention, the decoder can select the input words that matter for the output at this moment. At t=1 it attends to "Python program print". At t=2 it generates the bracket, at t=3 the quote token, at t=4 it attends to "hello", and at t=5 it puts heavy weight on "world".
- This way the encoder no longer has to pack all information into one context vector.

At each step the decoder puts most weight on the source word it needs

Attention weights for each output token over the seven encoder vectors

Output token	h1 start	h2 Python	h3 program	h4 to	h5 print	h6 hello	h7 world	Sum
t=1: print	0.02	0.12	0.06	0.02	0.72	0.03	0.03	1.00
t=2: (	0.02	0.05	0.05	0.05	0.65	0.10	0.08	1.00
t=3: "	0.02	0.03	0.03	0.04	0.30	0.38	0.20	1.00
t=4: hello	0.01	0.02	0.02	0.02	0.05	0.83	0.05	1.00
t=5: world	0.01	0.01	0.02	0.02	0.04	0.12	0.78	1.00
t=6: "	0.01	0.01	0.01	0.02	0.03	0.20	0.72	1.00
t=7:)	0.02	0.03	0.03	0.04	0.50	0.15	0.23	1.00

Outlined cell = highest weight for that step. Each row sums to 1.
Illustrative values for the lecture example, not from a trained model.

attention weights per output token · illustrative values

Note from the lecture: the video has a typo at 02:32 where one equation is referred to by the wrong name. The heatmap numbers above are illustrative, not from a trained model.

Part 4: Adaptive selection of vectors

The consolidated idea: the encoder keeps a sequence of vectors, and the decoder chooses from them adaptively.

- At each time point in the decoder, the model can attend to different vectors at different points in time.
- So the encoder no longer has to compress all the information into one fixed-length vector, which is a tricky and difficult process.
- The model does not try to encode the whole input sentence into a single fixed-length vector. It encodes the input into a sequence of vectors.
- The decoder then chooses a subset of these vectors adaptively. Adaptively means it does not take the same subset every time. If it did, the decoder's power would be taken away. It selects whatever is required at each time step.
- This frees the neural network from squashing all the information of a source sentence into a fixed-length vector, regardless of the sentence length.
- There is no longer a need for one vector that represents everything about the input. The individual vectors h1, h2, h3, h4 and so on are all available, and the decoder is free to select the subset it wants at each step.

- That resolves the issue raised in Part 1: it matters most when the input sentence is very long.

Part 5: How attention works at each step

The encoder side

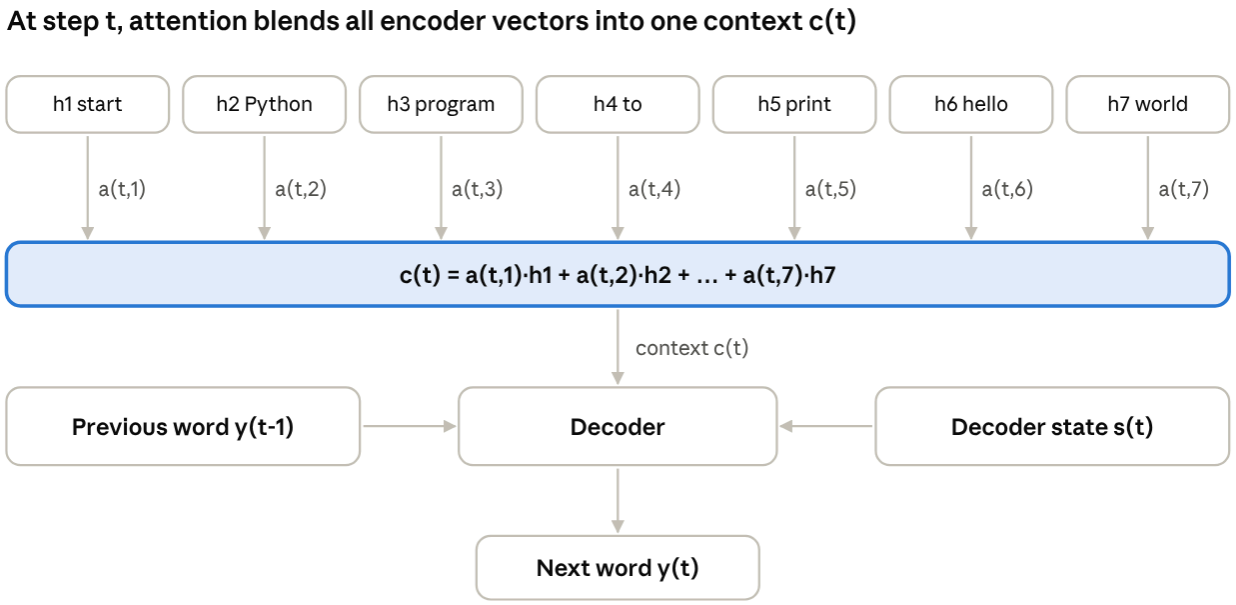
- The research paper uses a bidirectional RNN in the encoder. You can ignore that detail and focus on the key point: at each point in time, the encoder produces updated vectors h1, h2, h3 up to hT.
- Each vector represents its input word after incorporating information from the surrounding words, so it acts as a contextual embedding of that word.

The decoder side

- In the standard model, at each decoding step the decoder looks at the previous word y(t-1), the current state s(t), and a single context from the encoder, to generate the next word y(t).
- With attention, at time step t the decoder looks at all the encoder vectors h1 to hT and puts a weight on each: a(t,1), a(t,2), up to a(t,T).
- These weights are the relative importance of each vector at step t. Each is a number between 0 and 1. For example, a(t,1) says how important h1 is for the decoder at step t, and a(t,2) says how important h2 is.
- This is the alignment process: the decoder first computes the relative importance of each encoder vector, then combines them with a convex combination, a weighted sum:

```
c_t = a_{t1} h_1 + a_{t2} h_2 + \dots + a_{tT} h_T
```

- The result c(t) is the context at step t. Because the weights are recomputed at each step, the context differs from step to step, and it favours whichever vectors matter at that moment.



one decoder step with attention · schematic

Mapping it to the example

- Encoding "Python program to print hello world": h1 is the initial vector, h2 comes from "Python", h3 from "program", then "to", "print", "hello" and "world" give h4 to h7. That is seven vectors in total (h7 is correct, not h8), all produced by the same RNN function.
- The encoder hands h1 to h7 to the decoder, which translates them into the Python program.
- At t=1, before it writes anything, the decoder must decide a(1,1), a(1,2) up to a(1,7): the weight on each of h1 to h7.
- In an ideal, well-trained model the largest weight is a(1,5), on the vector for "print", and it is close to 1.
- Some attention also goes to "Python", because the language matters: in other languages the command might be printf.
- A smaller amount goes to "program". The weights on "hello" and "world" are above zero but close to zero, because what to print comes later.
- These are the weights at time step 1 only. Note the first subscript. If the same weights were used at every step, the model would not learn well.
- The lecture leaves an exercise: work out which weights should be set to generate "hello" and which to generate "world". The heatmap in Part 3 shows one answer.

Symbols used in this guide

Symbol	Meaning
h1 … hT	Encoder vectors (contextual embeddings), one per input position
C	The single fixed context vector of the standard model
c(t)	Context vector at decoder step t, the weighted sum of the h vectors
a(t,i)	Weight on encoder vector h(i) at step t, between 0 and 1
F	Encoder function, a matrix operation applied at each encoder state
G	Decoder function, a matrix operation applied at each decoder state
a	Alignment function that produces the weights
y(t) and s(t)	Word generated at step t and decoder state at step t

Part 6: Training the attention model

One loss trains the encoder, the alignment function and the decoder together.

Data

- As in any machine learning application, these models run on data, and a lot of it.
- To translate from language A to B there is no shortage of resources. Build a table with two columns: a sentence in language A and its correct translation in language B.
- The dataset holds M such examples. Creating it is not difficult.

Language A sentence	Language B translation
Python program to print hello world	print("hello world")
Sentence 2	Translated sentence 2
…	…

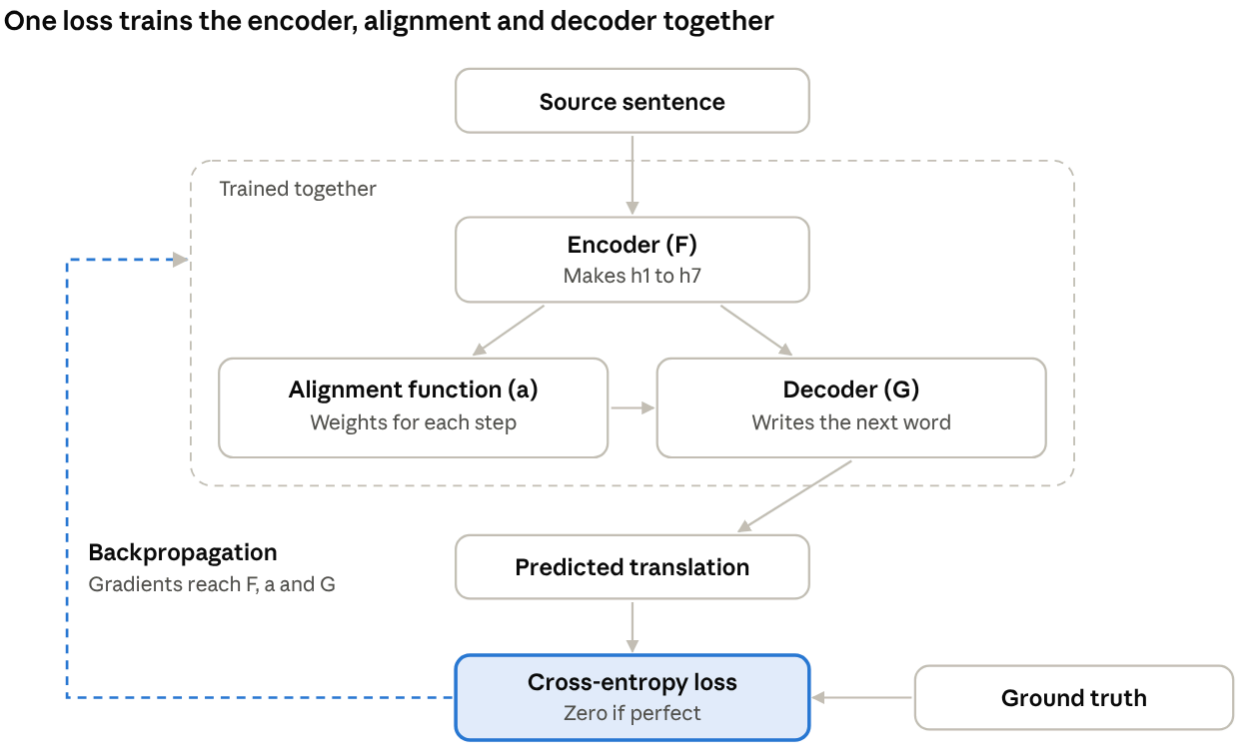
Language A sentence	Language B translation
Sentence M	Translated sentence M

Loss

- The whole encoder-decoder with attention is trained on a loss function, as with most machine learning algorithms: define a loss and ask the model to minimise it.
- The loss is the cross-entropy between the ground truth from the dataset and what the model predicts.
- If the model predicts exactly what it should, the loss is exactly 0. Otherwise it incurs a loss, and the objective is to minimise it.

What gets optimised

- The encoder function F: a matrix applied at each encoder state.
- The decoder function G: a matrix operation applied at each decoder state.
- The alignment function a, which learns the weights needed at each time step.
- All three are learned together. Even if F and G are optimal, the output stays suboptimal when the model has not learned to place the right weights at each step.
- When the loss is backpropagated, the gradient flows through all three components. They are constantly improved toward one goal: minimising the cross-entropy between the ground-truth translation and the model's translation.



training loop of the attention model · schematic

Seeing it work: English to French

- In the example from the research paper, highlighted areas show which English words the model attended to when producing each French word.
- For the French word for "European", a well-trained model focuses on "European" and "economic". For the word for "economic", it focuses on "economic" and also "zone", and so on.
- The model does not focus on the same word at every step. At each step it attends to whichever source words are required then.
- This ability is not there from the beginning. Early in training the attention is not optimal. As training drives the loss down, the model adjusts the weights, functions and matrices until it does this well.
- That is the advantage of attaching the attention module to the encoder and decoder, and it is more efficient than the standard encoder-decoder.

Summary and what comes next

1. The standard encoder-decoder compresses the whole input into one fixed-length vector, which struggles on long sentences.
2. Attention lets the decoder look at all encoder vectors and select what it needs at each step, so the context changes at every step.

3. The decoder computes weights a(t,i) between 0 and 1 and takes their weighted sum of the h vectors as the context c(t).
4. The encoder function F, the decoder function G and the alignment function a are trained together by minimising cross-entropy loss on pairs of source and target sentences.
5. A trained model attends to the source words that matter for each target word, as the English to French example shows.

Next in the course: a Python demonstration of an encoder-decoder with attention performing machine translation.